

LESSON 1

Python Essentials for Coding Interviews

Everything you need to write clean Python in a 45-minute interview.
No fluff. Only what matters.

- 1.1 Variables, Data Types & Type Conversion
- 1.2 Lists (Arrays) - Complete Guide
- 1.3 Dictionaries (Hash Maps)
- 1.4 Strings - All Operations
- 1.5 Sets & Tuples
- 1.6 Loops - for, while, enumerate, zip
- 1.7 Conditions & Logical Operators
- 1.8 Functions - def, lambda, *args, **kwargs
- 1.9 List Comprehensions
- 1.10 Important Built-in Functions
- 1.11 Common Mistakes to Avoid
- 1.12 Practice Problems with Solutions

1.1 Variables, Data Types & Type Conversion

Python is dynamically typed — you don't declare types. The interpreter figures it out.

Basic Types

```
# Integer - whole numbers
age = 25
count = -3
big_number = 1_000_000    # underscores for readability (= 1000000)

# Float - decimal numbers
pi = 3.14159
temperature = -40.5
tiny = 1e-6               # scientific notation (= 0.000001)

# String - text (single or double quotes both work)
name = "Anushka"
greeting = 'Hello'
multiline = """This is
a multiline
string"""

# Boolean - True or False (capital T and F!)
is_valid = True
has_error = False

# None - represents "nothing" / no value
result = None
```

Checking Types

```
type(42)           # <class 'int'>
type(3.14)         # <class 'float'>
type("hello")     # <class 'str'>
type(True)        # <class 'bool'>
type(None)        # <class 'NoneType'>

# Check if something is a specific type
isinstance(42, int)    # True
isinstance("hi", str)  # True
isinstance(3.14, float) # True
```

Type Conversion (Very Important)

Interviewers expect you to handle conversions smoothly:

```

# String to Number
int("42")          # 42
int("42.7")        # ERROR! Can't directly convert decimal string to int
int(float("42.7")) # 42 (convert to float first, then int)
float("3.14")       # 3.14

# Number to String
str(42)            # "42"
str(3.14)          # "3.14"

# Number conversions
int(3.7)           # 3 (truncates, does NOT round)
int(3.2)           # 3
float(5)           # 5.0
round(3.7)         # 4 (this one rounds)
round(3.14159, 2) # 3.14 (round to 2 decimal places)

# Boolean conversions (tricky!)
bool(0)            # False
bool(1)            # True
bool(-5)           # True (any non-zero number is True)
bool("")           # False (empty string)
bool("hello")      # True (non-empty string)
bool([])           # False (empty list)
bool([1,2])        # True (non-empty list)
bool(None)         # False

```

>> INTERVIEW TIP: "Falsy" values in Python are: 0, 0.0, "", [], {}, set(), None, False
Everything else is "Truthy". This matters in if-conditions!

Arithmetic Operators

```

10 + 3    # 13    addition
10 - 3    # 7     subtraction
10 * 3    # 30    multiplication
10 / 3    # 3.333 division (ALWAYS returns float)
10 // 3   # 3     integer division (floor)
10 % 3    # 1     modulo (remainder) *** VERY IMPORTANT ***
10 ** 3   # 1000  exponent (10^3)

# Modulo is used constantly in interviews:
# Check if even:    num % 2 == 0
# Check if odd:     num % 2 != 0
# Check divisible:  num % k == 0
# Get last digit:   num % 10
# Wrap around:      index % length

```

Comparison & Assignment Operators

```
# Comparison (return True/False)
a == b    # equal
a != b    # not equal
a > b     # greater than
a < b     # less than
a >= b    # greater than or equal
a <= b    # less than or equal

# Shorthand assignment
x += 5     # same as x = x + 5
x -= 3     # same as x = x - 3
x *= 2     # same as x = x * 2
x //= 2    # same as x = x // 2

# Swap two variables (VERY Pythonic, use in interviews)
a, b = b, a # swap! No temp variable needed

# Multiple assignment
x, y, z = 1, 2, 3
a = b = c = 0 # all set to 0
```

1.2 Lists (Arrays) - Complete Guide

Lists are the most important data structure in Python interviews. Master every operation.

Creating Lists

```
# Empty list
nums = []
nums = list()          # same thing

# With values
nums = [1, 2, 3, 4, 5]
mixed = [1, "hello", True, 3.14]    # can mix types (but rarely do)
nested = [[1,2], [3,4], [5,6]]      # 2D list (matrix)

# Create list of N zeros
zeros = [0] * 5          # [0, 0, 0, 0, 0]
grid = [[0]*3 for _ in range(3)]    # 3x3 grid of zeros
# WARNING: [[0]*3]*3 creates shared references! Always use for loop.

# Convert other things to list
list("hello")            # ['h', 'e', 'l', 'l', 'o']
list(range(5))           # [0, 1, 2, 3, 4]
list(range(2, 8))        # [2, 3, 4, 5, 6, 7]
list(range(0, 10, 2))     # [0, 2, 4, 6, 8]
```

Accessing Elements (Indexing)

```
nums = [10, 20, 30, 40, 50]
#      0   1   2   3   4      <- positive index
#      -5  -4  -3  -2  -1     <- negative index

nums[0]      # 10 (first element)
nums[1]      # 20 (second element)
nums[-1]     # 50 (LAST element) *** USE THIS A LOT ***
nums[-2]     # 40 (second to last)

# Accessing 2D list (matrix)
matrix = [[1,2,3], [4,5,6], [7,8,9]]
matrix[0]    # [1, 2, 3] (first row)
matrix[0][1] # 2 (row 0, column 1)
matrix[2][2] # 9 (row 2, column 2)
```

Slicing (Very Powerful)

```

nums = [10, 20, 30, 40, 50]

# Syntax: list[start:end:step]
# start = included, end = EXCLUDED

nums[1:3]      # [20, 30]          index 1 to 2
nums[:3]       # [10, 20, 30]     first 3 elements
nums[2:]       # [30, 40, 50]     from index 2 to end
nums[:]        # [10, 20, 30, 40, 50] full copy
nums[::2]      # [10, 30, 50]     every 2nd element
nums[::-1]     # [50, 40, 30, 20, 10] REVERSED! *** REMEMBER THIS ***
nums[1:4]      # [20, 30, 40]     index 1, 2, 3

# Slicing never throws IndexError (even with wrong indices)
nums[10:]      # [] (empty, no error)
nums[:100]     # [10, 20, 30, 40, 50] (just gives everything)

```

Adding Elements

```

nums = [1, 2, 3]

# Add to end - O(1)
nums.append(4)      # [1, 2, 3, 4]

# Add at specific position - O(n)
nums.insert(0, 99)   # [99, 1, 2, 3, 4] insert 99 at index 0
nums.insert(2, 88)   # [99, 1, 88, 2, 3, 4] insert 88 at index 2

# Add multiple elements to end - O(k)
nums.extend([5, 6, 7]) # [99, 1, 88, 2, 3, 4, 5, 6, 7]
# Same as: nums += [5, 6, 7]

# Concatenation (creates NEW list)
combined = [1, 2] + [3, 4] # [1, 2, 3, 4]

```

Removing Elements

```

nums = [10, 20, 30, 20, 40]

# Remove and return last element - O(1)
last = nums.pop()    # last = 40, nums = [10, 20, 30, 20]

# Remove and return element at index - O(n)
second = nums.pop(1) # second = 20, nums = [10, 30, 20]

# Remove first occurrence of a VALUE - O(n)
nums.remove(30)       # nums = [10, 20]
# WARNING: raises ValueError if value not found!

# Remove by index (no return value)
del nums[0]           # nums = [20]

# Clear everything
nums.clear()          # nums = []

```

Searching & Checking

```
nums = [3, 1, 4, 1, 5, 9]

# Check if element exists - O(n)
3 in nums          # True
7 in nums          # False
7 not in nums      # True

# Find index of first occurrence - O(n)
nums.index(1)      # 1 (first 1 is at index 1)
# WARNING: raises ValueError if not found!

# Count occurrences
nums.count(1)      # 2 (appears twice)

# Length
len(nums)          # 6

# Min, Max, Sum
min(nums)          # 1
max(nums)          # 9
sum(nums)          # 23

# Check if list is empty
if not nums:       # True if empty
    print("empty!")
if nums:           # True if NOT empty
    print("has items!")
```

Sorting (Extremely Important for Interviews)

```
nums = [3, 1, 4, 1, 5, 9]

# Sort in-place (modifies original) - O(n log n)
nums.sort()        # [1, 1, 3, 4, 5, 9]
nums.sort(reverse=True) # [9, 5, 4, 3, 1, 1]

# Return NEW sorted list (original unchanged)
original = [3, 1, 4]
new_sorted = sorted(original) # new_sorted = [1, 3, 4]
# original is still [3, 1, 4]

# Sort by custom key
words = ["banana", "pie", "apple", "hi"]
words.sort(key=len) # ["hi", "pie", "apple", "banana"] by length

pairs = [(2,"b"), (1,"a"), (3,"c")]
pairs.sort()        # [(1,"a"), (2,"b"), (3,"c")] by first element
pairs.sort(key=lambda x: x[1]) # sort by second element

# Sort by multiple criteria
students = [("Alice", 90), ("Bob", 85), ("Carol", 90)]
students.sort(key=lambda x: (-x[1], x[0])) # by grade desc, then name asc
# [("Alice", 90), ("Carol", 90), ("Bob", 85)]
```

Reversing

```
nums = [1, 2, 3, 4, 5]

# In-place
nums.reverse()          # [5, 4, 3, 2, 1]

# New reversed list
rev = nums[::-1]        # creates new list
rev = list(reversed(nums)) # same thing

# Reverse a portion only
nums = [1, 2, 3, 4, 5]
nums[1:4] = nums[1:4][::-1] # [1, 4, 3, 2, 5] reverse index 1-3
```

```
!! COMMON MISTAKE: .sort() returns None, not the sorted list!
WRONG: result = nums.sort()      # result = None!
RIGHT: nums.sort()               # modifies nums in-place
RIGHT: result = sorted(nums)     # returns new sorted list
```


1.3 Dictionaries (Hash Maps)

Key-value pairs with $O(1)$ lookup. The most powerful tool for interviews.

Creating & Basic Operations

```
# Creating
d = {}                # empty
d = dict()            # same
d = {"name": "Anushka", "age": 24} # with values

# Adding / Updating
d["city"] = "Pune"    # add new key
d["age"] = 25         # update existing key

# Accessing
d["name"]             # "Anushka"
# d["phone"]          # KeyError! Key doesn't exist
d.get("phone")        # None (no error)
d.get("phone", "N/A") # "N/A" (custom default)

# CRITICAL PATTERN for interviews:
d.get(key, 0) + 1     # count pattern: default to 0, add 1
```

Checking, Removing, Looping

```
d = {"a": 1, "b": 2, "c": 3}

# Check if key exists
"a" in d                # True
"z" in d                # False
"z" not in d            # True

# Remove
del d["a"]              # remove key "a"
val = d.pop("b")        # remove and return value (val = 2)
val = d.pop("z", None)  # remove if exists, else None (no error)

# Looping (KNOW ALL THREE)
for key in d:           # loop through keys
    print(key)

for key, value in d.items(): # loop through key-value pairs
    print(key, value)

for value in d.values():   # loop through values only
    print(value)

# Get all keys/values as lists
keys = list(d.keys())     # ["c"]
vals = list(d.values())   # [3]

# Length
len(d)                   # 1
```

Advanced Dictionary Patterns (Interview Gold)

```
# Pattern 1: Counting (MOST COMMON)
text = "hello world"
freq = {}
for char in text:
    freq[char] = freq.get(char, 0) + 1
# {'h':1, 'e':1, 'l':3, 'o':2, ' ':1, 'w':1, 'r':1, 'd':1}

# Pattern 2: Grouping
words = ["eat", "tea", "ate", "bat"]
groups = {}
for word in words:
    key = "".join(sorted(word))
    if key not in groups:
        groups[key] = []
    groups[key].append(word)
# {"aet": ["eat", "tea", "ate"], "abt": ["bat"]}

# Pattern 3: Two-sum complement lookup
seen = {}
for i, num in enumerate(nums):
    complement = target - num
    if complement in seen:
        return [seen[complement], i]
    seen[num] = i

# Pattern 4: Index mapping
nums = [10, 20, 30, 40]
index_map = {val: i for i, val in enumerate(nums)}
# {10: 0, 20: 1, 30: 2, 40: 3}
index_map[30]  # 2 (instant lookup!)

# Pattern 5: defaultdict (auto-creates default values)
from collections import defaultdict
freq = defaultdict(int)  # missing keys default to 0
for char in "hello":
    freq[char] += 1  # no need for .get()!

groups = defaultdict(list)  # missing keys default to []
groups["team_a"].append("Alice")  # auto-creates the list
```

1.4 Strings - All Operations

Strings in Python are **immutable** — you cannot change them after creation. Every 'modification' creates a new string.

```

s = "Hello, World!"

# ■■ Accessing ■■
s[0]          # "H"
s[-1]         # "!"
s[7:12]       # "World"
s[:5]         # "Hello"
len(s)        # 13

# ■■ Case ■■
s.upper()     # "HELLO, WORLD!"
s.lower()     # "hello, world!"
s.capitalize() # "Hello, world!"
s.title()     # "Hello, World!"
s.swapcase()  # "hELLO, wORLD!"

# ■■ Checking ■■
"Hello" in s   # True
s.startswith("Hello") # True
s.endswith("!") # True
"123".isdigit() # True
"abc".isalpha() # True
"abc123".isalnum() # True (letters + digits)
"   ".isspace() # True

# ■■ Finding ■■
s.find("World") # 7 (index where found)
s.find("xyz")   # -1 (not found, NO error)
s.index("World") # 7 (same but raises ValueError if not found)
s.count("l")    # 3

# ■■ Modifying (all return NEW string) ■■
s.replace("World", "Python") # "Hello, Python!"
"  hello  ".strip()         # "hello" (remove whitespace)
"  hello  ".lstrip()        # "hello " (left only)
"  hello  ".rstrip()        # "  hello" (right only)
"***hello***".strip("")     # "hello"

# ■■ Splitting & Joining (VERY IMPORTANT) ■■
"hello world foo".split()   # ["hello", "world", "foo"]
"a,b,c".split(",")         # ["a", "b", "c"]
"hello".split("")          # ERROR! Use list("hello") instead

" ".join(["hello", "world"]) # "hello world"
",".join(["a", "b", "c"])    # "a,b,c"
"".join(["h", "i"])          # "hi"

# ■■ Formatting (f-strings) ■■
name = "Anushka"
age = 24
f"My name is {name} and I am {age} years old"
f"Result: {10 + 5}"          # "Result: 15"
f"Pi is {3.14159:.2f}"      # "Pi is 3.14" (2 decimal places)
f"Count: {42:05d}"          # "Count: 00042" (pad with zeros)

```

String to List and Back (Interview Essential)

```
# String -> List of characters
chars = list("hello")          # ['h', 'e', 'l', 'l', 'o']

# List of characters -> String
"".join(chars)                 # "hello"

# WHY? Strings are immutable. To modify individual chars:
s = "hello"
chars = list(s)                # convert to list
chars[0] = "H"                 # modify
s = "".join(chars)             # back to string: "Hello"

# Reverse a string
"hello"[::-1]                  # "olleh"
```

```
!! IMMUTABILITY TRAP:
s = "hello"
s[0] = "H"    # TypeError! Strings can't be modified!
CORRECT: s = "H" + s[1:]    # Creates new string "Hello"
```

1.5 Sets & Tuples

Sets - Unique elements, O(1) lookup

```
# Creating
my_set = {1, 2, 3}
my_set = set()          # empty set (NOT {} which is empty dict!)
my_set = set([1, 2, 2, 3, 3]) # {1, 2, 3} (auto removes duplicates)

# Operations
my_set.add(4)            # add element
my_set.remove(2)         # remove (error if not found)
my_set.discard(99)       # remove (NO error if not found)

# Checking (O(1) - FAST!)
3 in my_set              # True

# Set math (interview useful)
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
a & b                    # {3, 4}          intersection (common elements)
a | b                    # {1,2,3,4,5,6}   union (all elements)
a - b                    # {1, 2}          difference (in a but not b)
a ^ b                    # {1, 2, 5, 6}   symmetric difference

# INTERVIEW TRICK: Remove duplicates from list
nums = [1, 3, 2, 3, 1, 4]
unique = list(set(nums)) # [1, 2, 3, 4] (order may vary)

# INTERVIEW TRICK: Find common elements
list1 = [1, 2, 3, 4]
list2 = [3, 4, 5, 6]
common = list(set(list1) & set(list2)) # [3, 4]
```

Tuples - Immutable lists

```

# Creating
t = (1, 2, 3)
t = 1, 2, 3          # parentheses optional
single = (1,)        # single-element tuple NEEDS comma

# Access (like lists)
t[0]                 # 1
t[-1]                # 3

# CANNOT modify
# t[0] = 99          # TypeError!

# WHY use tuples?
# 1. Can be dictionary keys (lists can't)
d = {(0,1): "start", (2,3): "end"} # grid coordinates as keys

# 2. Can be added to sets
visited = set()
visited.add((0, 1))   # track visited grid positions

# 3. Unpacking
x, y = (3, 4)         # x=3, y=4
a, b, c = (1, 2, 3)   # a=1, b=2, c=3

```

1.6 Loops - for, while, enumerate, zip

for loops

```
# Basic range
for i in range(5):          # i = 0, 1, 2, 3, 4
    print(i)

for i in range(2, 7):       # i = 2, 3, 4, 5, 6
    print(i)

for i in range(0, 10, 2):   # i = 0, 2, 4, 6, 8 (step=2)
    print(i)

for i in range(5, 0, -1):   # i = 5, 4, 3, 2, 1 (counting DOWN)
    print(i)

# Loop through a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# Loop through a string
for char in "hello":
    print(char)             # h, e, l, l, o

# Loop through a dictionary
d = {"a": 1, "b": 2}
for key in d:
    print(key, d[key])
for key, val in d.items():
    print(key, val)
```

enumerate() - Loop with Index (USE THIS!)

```
# Instead of this ugly pattern:
for i in range(len(nums)):
    print(i, nums[i])

# Do THIS:
nums = [10, 20, 30]
for i, num in enumerate(nums):
    print(i, num)

# Output:
# 0 10
# 1 20
# 2 30

# Start from different index
for i, num in enumerate(nums, start=1):
    print(i, num)          # 1 10, 2 20, 3 30
```

zip() - Loop Through Multiple Lists Together


```

names = ["Alice", "Bob", "Carol"]
scores = [90, 85, 92]

for name, score in zip(names, scores):
    print(f"{name}: {score}")
# Alice: 90
# Bob: 85
# Carol: 92

# Create dict from two lists
name_score = dict(zip(names, scores))
# {"Alice": 90, "Bob": 85, "Carol": 92}

```

while loops

```

# Basic while
i = 0
while i < 5:
    print(i)
    i += 1      # DON'T FORGET THIS or infinite loop!

# Common pattern: process until empty
stack = [1, 2, 3]
while stack:
    item = stack.pop()
    print(item)  # 3, 2, 1

# Two pointer pattern
left, right = 0, len(nums) - 1
while left < right:
    # ... do something ...
    left += 1
    right -= 1

```

break, continue, else

```

# break - exit loop entirely
for num in [1, 2, 3, 4, 5]:
    if num == 3:
        break      # stops at 3, doesn't process 4, 5
    print(num)     # prints 1, 2

# continue - skip current iteration
for num in [1, 2, 3, 4, 5]:
    if num == 3:
        continue   # skips 3
    print(num)     # prints 1, 2, 4, 5

# for-else (runs if loop completed WITHOUT break)
for num in [2, 4, 6]:
    if num % 2 != 0:
        print("Found odd!")
        break
else:
    print("All even!")  # This runs because no break happened

```

1.7 Conditions & Logical Operators

```
# Basic if/elif/else
x = 15
if x > 20:
    print("big")
elif x > 10:
    print("medium")    # this runs
elif x > 5:
    print("small")     # skipped (already matched above)
else:
    print("tiny")

# Logical operators: and, or, not
if x > 10 and x < 20:    # both must be True
    print("between 10 and 20")

if x < 5 or x > 10:      # at least one must be True
    print("outside 5-10")

if not (x == 15):        # negation
    print("not 15")

# Chained comparison (Pythonic!)
if 10 < x < 20:          # same as: x > 10 and x < 20
    print("in range")

# Ternary (one-line if)
result = "even" if x % 2 == 0 else "odd"
bigger = max(a, b)        # built-in alternative to if/else

# None checking
if result is None:       # use "is" for None, not "=="
    print("no result")
if result is not None:
    print("got result")
```

1.8 Functions - def, lambda, *args, **kwargs

```
# Basic function
def add(a, b):
    return a + b

result = add(3, 5)    # 8

# Default parameters
def greet(name, msg="Hello"):
    return f"{msg}, {name}!"

greet("Anushka")      # "Hello, Anushka!"
greet("Anushka", "Hi") # "Hi, Anushka!"

# Multiple return values
def min_max(nums):
    return min(nums), max(nums)

lo, hi = min_max([3, 1, 7, 2])    # lo=1, hi=7

# Return early (guard clause)
def find_first_negative(nums):
    for num in nums:
        if num < 0:
            return num    # exits immediately
    return None           # no negative found

# Lambda (anonymous one-line function)
square = lambda x: x ** 2
square(5)    # 25

# Main use: as sort key
pairs = [(1,"b"), (2,"a"), (3,"c")]
pairs.sort(key=lambda x: x[1])    # sort by second element

# *args - variable number of arguments
def add_all(*args):
    return sum(args)
add_all(1, 2, 3, 4)    # 10

# **kwargs - keyword arguments as dictionary
def print_info(**kwargs):
    for key, val in kwargs.items():
        print(f"{key}: {val}")
print_info(name="Anushka", age=24)
```

```
!! SCOPE TRAP:
x = 10
def change():
    x = 20    # This creates a LOCAL x, doesn't change outer x!
change()
print(x)    # Still 10!
```

1.9 List Comprehensions (Pythonic & Impressive)

```
# Basic: [expression for item in iterable]
squares = [x**2 for x in range(5)]      # [0, 1, 4, 9, 16]

# With condition: [expr for item in iter if condition]
evens = [x for x in range(10) if x % 2 == 0] # [0, 2, 4, 6, 8]

# Transform strings
words = ["Hello", "WORLD"]
lower = [w.lower() for w in words]      # ["hello", "world"]

# Flatten 2D list
matrix = [[1,2], [3,4], [5,6]]
flat = [x for row in matrix for x in row] # [1,2,3,4,5,6]

# Dictionary comprehension
nums = [1, 2, 3, 4]
sq_dict = {x: x**2 for x in nums}       # {1:1, 2:4, 3:9, 4:16}

# Set comprehension
unique_lengths = {len(w) for w in ["hi","hello","hey"]} # {2, 5, 3}

# Generator expression (memory efficient, uses () not [])
total = sum(x**2 for x in range(1000000)) # doesn't store entire list
```

1.10 Important Built-in Functions

```
# ■■ Math ■■
abs(-5)           # 5
min(3, 1, 4)      # 1
max(3, 1, 4)      # 4
min([3, 1, 4])    # 1 (also works on lists)
sum([1, 2, 3])    # 6
round(3.14159, 2) # 3.14
pow(2, 10)        # 1024 (same as 2**10)
divmod(17, 5)     # (3, 2) -> quotient=3, remainder=2

float("inf")       # infinity (useful for min-finding)
float("-inf")      # negative infinity

# ■■ Conversion ■■
int(), float(), str(), bool(), list(), tuple(), set(), dict()
chr(65)           # "A" (number to character)
ord("A")          # 65 (character to number)
bin(10)           # "0b1010" (binary)
hex(255)          # "0xff"

# ■■ Iteration ■■
range(n)          # 0 to n-1
enumerate(list)   # (index, value) pairs
zip(list1, list2) # pair up elements
reversed(list)    # reverse iterator
sorted(list)      # new sorted list

# ■■ Higher-order ■■
map(func, list)   # apply func to each element
list(map(int, ["1","2","3"])) # [1, 2, 3]
list(map(str, [1, 2, 3]))    # ["1", "2", "3"]

filter(func, list) # keep elements where func returns True
list(filter(lambda x: x>0, [-1,2,-3,4])) # [2, 4]

# ■■ Checking ■■
all([True, True, False]) # False (ALL must be True)
any([True, False, False]) # True (ANY one True is enough)
all(x > 0 for x in [1,2,3]) # True (all positive)
any(x < 0 for x in [1,-2,3]) # True (at least one negative)

# ■■ Useful from collections ■■
from collections import Counter, defaultdict, deque

Counter("banana") # Counter({'a':3, 'n':2, 'b':1})
Counter([1,2,2,3,3,3]) # Counter({3:3, 2:2, 1:1})
Counter("banana").most_common(2) # [('a',3), ('n',2)]

from collections import deque
q = deque() # double-ended queue
q.append(1) # add to right
q.appendleft(0) # add to left
q.pop() # remove from right
q.popleft() # remove from left (O(1)!)
# List.pop(0) is O(n), deque.popleft() is O(1) -> use for BFS!
```

1.11 Common Mistakes to Avoid in Interviews

Modifying list while iterating

```
# WRONG
for item in my_list:
    if item < 0:
        my_list.remove(item) # unpredictable!

# RIGHT
my_list = [x for x in my_list if x >= 0]
```

Using = instead of ==

```
# WRONG
if x = 5:      # SyntaxError!

# RIGHT
if x == 5:     # comparison
```

Off-by-one errors in range

```
# range(5) gives 0,1,2,3,4 (NOT 5)
# range(1,5) gives 1,2,3,4 (NOT including 5)
# To include 5: range(1, 6)
```

Integer division surprise

```
# In Python 3:
7 / 2    # 3.5 (float division)
7 // 2   # 3   (integer division)
# Use // when you want integer result
```

Mutable default argument

```
# WRONG
def add_item(item, lst=[]):
    lst.append(item) # lst is SHARED across calls!
    return lst

# RIGHT
def add_item(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst
```

Forgetting return in function

```
# WRONG
def add(a, b):
    a + b          # computes but doesn't return!

# RIGHT
def add(a, b):
    return a + b
```

String concatenation in loop (slow)

```
# SLOW -  $O(n^2)$ 
result = ""
for word in words:
    result += word + " "

# FAST -  $O(n)$ 
result = " ".join(words)
```

1.12 Practice Problems with Solutions

Problem 1: Sum of Even Numbers

Given a list, return the sum of all even numbers.

```
def sum_even(nums):
    total = 0
    for num in nums:
        if num % 2 == 0:
            total += num
    return total

# One-liner:
def sum_even(nums):
    return sum(x for x in nums if x % 2 == 0)

# Test:
sum_even([1, 2, 3, 4, 5, 6]) # 12 (2+4+6)
```

```
nums = [1, 2, 3, 4, 5, 6]
1 -> odd, skip
2 -> even, total = 0 + 2 = 2
3 -> odd, skip
4 -> even, total = 2 + 4 = 6
5 -> odd, skip
6 -> even, total = 6 + 6 = 12
Return 12
```

Problem 2: Find Maximum Element

Without using max(), find the largest number in a list.

```
def find_max(nums):
    if not nums:
        return None

    maximum = nums[0] # start with first element
    for i in range(1, len(nums)):
        if nums[i] > maximum:
            maximum = nums[i]
    return maximum

# Alternative using float("-inf")
def find_max(nums):
    maximum = float("-inf") # smaller than any number
    for num in nums:
        if num > maximum:
            maximum = num
    return maximum

find_max([3, 7, 2, 9, 1]) # 9
```

Problem 3: Count Words in a String


```
def count_words(s):
    words = s.split()    # split on whitespace
    return len(words)

count_words("Hello World")    # 2
count_words("  Hello  World  ")    # 2 (split handles extra spaces)
```

Problem 4: Reverse Words in a String

```
def reverse_words(s):
    words = s.split()    # ["Hello", "World"]
    words.reverse()      # ["World", "Hello"]
    return " ".join(words)    # "World Hello"

# One-liner:
def reverse_words(s):
    return " ".join(s.split()[::-1])

reverse_words("the sky is blue")    # "blue is sky the"
```

Problem 5: Find Second Largest

```
def second_largest(nums):
    if len(nums) < 2:
        return None

    first = second = float("-inf")

    for num in nums:
        if num > first:
            second = first    # old first becomes second
            first = num       # new first
        elif num > second and num != first:
            second = num

    return second if second != float("-inf") else None

second_largest([5, 2, 8, 1, 9])    # 8
```

```
nums = [5, 2, 8, 1, 9]
first=-inf, second=-inf

num=5: 5 > -inf -> second=-inf, first=5
num=2: 2 < 5, but 2 > -inf -> second=2
num=8: 8 > 5 -> second=5, first=8
num=1: 1 < 8, 1 < 5 -> skip
num=9: 9 > 8 -> second=8, first=9

Return second = 8
```

Problem 6: Character Frequency

```
def char_frequency(s):
    freq = {}
    for char in s:
        if char != " ": # skip spaces
            freq[char] = freq.get(char, 0) + 1
    return freq

# Using Counter (one line):
from collections import Counter
def char_frequency(s):
    return Counter(s.replace(" ", ""))

char_frequency("hello") # {'h':1, 'e':1, 'l':2, 'o':1}
```

Problem 7: FizzBuzz (Classic Interview Warm-up)

```
def fizzbuzz(n):
    for i in range(1, n + 1):
        if i % 15 == 0: # divisible by both 3 AND 5
            print("FizzBuzz")
        elif i % 3 == 0:
            print("Fizz")
        elif i % 5 == 0:
            print("Buzz")
        else:
            print(i)

# fizzbuzz(15) -> 1,2,Fizz,4,Buzz,Fizz,7,8,Fizz,Buzz,
#                11,Fizz,13,14,FizzBuzz
```

>> KEY INSIGHT: Check divisible by 15 FIRST (before 3 or 5).
Otherwise, 15 would match "Fizz" first and never reach "FizzBuzz".

Problem 8: Flatten a 2D List

```
# Method 1: Nested loop
def flatten(matrix):
    result = []
    for row in matrix:
        for item in row:
            result.append(item)
    return result

# Method 2: List comprehension
def flatten(matrix):
    return [item for row in matrix for item in row]

flatten([[1,2], [3,4], [5,6]]) # [1, 2, 3, 4, 5, 6]
```

End of Lesson 1. Next: Lesson 2 - Arrays & Two Pointers